📐 PagingTemplate

> **Unity C# | MVP + FSM + DataRepository | 키오스크 페이징 UI 프레임워크**

📃 프로젝트 요약

항목	내용
프로젝트명	PagingTemplate
개발 기간	2026.03
플랫폼	Windows (키오스크)
엔진	Unity
언어	C#
주요 역할	아키텍처 설계, 전체 구현
배포 환경	키오스크 전용 PC (단일 씬, 터치/마우스 입력)

🚀 프로젝트 소개

키오스크 환경에서 반복적으로 필요한 **페이징 화면 전환 UI**를 재사용 가능한 프레임워크 템플릿으로 구축한 프로젝트입니다. 새 키오스크 프로젝트 시작 시 View와 CSV 파일만 추가하면 즉시 동작하는 구조를 목표로 설계했습니다.

- 📑 **페이징 화면 전환**: Prev / Next / Home 버튼으로 화면 간 이동
- ⏰ **무입력 자동 복귀**: 설정 시간 경과 시 초기 화면으로 자동 복귀
- 📄 **CSV 기반 콘텐츠 관리**: 코드 수정 없이 외부 파일로 화면 콘텐츠 교체
- 🔌 **확장 용이한 구조**: View + State + CSV 3개 파일만 추가하면 새 화면 완성

> **MVP + FSM + DataRepository 패턴 조합**으로 관심사 분리와 확장성을 동시에 달성한 프로젝트입니다.

🎯 해결하려는 문제

문제	설명
반복 구현 비용	키오스크 프로젝트마다 유사한 화면 전환, 타임아웃, 데이터 바인딩 코드를 매번 새로 작성
View-로직 결합	MonoBehaviour에 UI 표시와 비즈니스 로직이 혼재되어 유지보수 어려움
콘텐츠 변경 시 코드 수정	텍스트·이미지 경로가 코드에 하드코딩되어 기획 변경마다 빌드 필요
화면 전환 스파게티	화면이 늘어날수록 전환 조건이 분산되어 흐름 파악 곤란

해결 전략:

- **MVP 패턴**으로 View(표시)와 State(로직)를 분리하여 독립 변경 가능
- **FSM 패턴**으로 화면 전환을 상태 기계로 모델링, 전환 로직을 State 안에 캡슐화
- **DataRepository 패턴**으로 콘텐츠를 CSV 외부 파일로 분리, 코드 수정 없이 교체
- **템플릿 구조**로 새 프로젝트에 복사 후 즉시 활용 가능

🧩 핵심 기능

🔀 페이징 화면 전환

Prev / Next / Home 버튼 기반 화면 이동. 각 화면(State)이 자신의 전환 규칙을 캡슐화하여 전환 흐름이 명확합니다.

⏰ 무입력 자동 복귀

마우스·터치·키보드 입력을 통합 감지하여, 설정 시간(기본 60초) 무입력 시 초기 화면으로 자동 복귀합니다. 일시정지/재개 API도 제공합니다.

📄 CSV 기반 데이터 관리

`DataConfig.json` 이 View별 CSV 파일을 선언적으로 매핑합니다. View당 복수 CSV 지원으로 기본 데이터 + 오버라이드 구조도 가능합니다.

🔦 4단계 화면 추가

View 생성 → State 생성 → NavigationManager 등록 → DataConfig.json 항목 추가. 정형화된 패턴으로 실수 없이 확장할 수 있습니다.

🛡 State Init 1회 보장

StateMachine이 `HashSet` 으로 Init 호출 이력을 추적하여, 화면 재진입 시 이벤트 중복 구독 버그를 원천 차단합니다.

🧠 시스템 아키텍처

```
┌──────────────────────────────────────┐
│              View Layer              │
│   BaseView ← StartView / ContentView / │
│               ResultView             │
│   (UI 표시, 버튼 이벤트 발행)          │
└──────────────────────────────────────┘
                  │
                  ▼ event Action
┌──────────────────────────────────────┐
│           State Layer (Presenter)    │
│   BaseState<TState,TView>            │
│   (이벤트 수신, BindView, GoTo<T>)     │
└──────────────────────────────────────┘
                  │
                  ▼ GoTo<T>()
┌──────────────────────────────────────┐
│           Navigation Layer           │
│   NavigationManager + StateMachine   │
│   + IdleManager                      │
└──────────────────────────────────────┘
                  │
                  ▼ GetData<TView>()
┌──────────────────────────────────────┐
│              Data Layer              │
│   DataRepository → CSVParser → PageData │
└──────────────────────────────────────┘
```

설계 목표:

- **관심사 분리**: View는 표시만, State는 로직만, Data는 저장만
- **단방향 의존**: 상위 레이어가 하위 레이어만 참조 (View → State ← Navigation ← Data)
- **확장성**: 새 화면 추가 시 기존 코드 수정 최소화
- **단순성**: 키오스크 특성에 맞게 비동기 처리 등 불필요한 복잡도 배제

🔄 화면 흐름

```
                ┌──────────────────────────┐
                │                          │
                ▼                          │
        ┌─────────┐   Next   ┌──────────┐   │
        │ Start   │ ────────▶ │ Content  │   │
        └─────────┘          └──────┬───┘   │
                ▲               │      │     │
                │          Prev/Home  Next │     │
                │               │      ▼     │
                │               │  ┌────────┐ │
                │               │  │ Result │ │
                │               │  └────┬───┘ │
                │               │  Prev │ Home │
                └───────────────┴───────┴──────┘

⏰ 어떤 화면에서든 무입력 타임아웃 → Start로 복귀
```

UX 특징:

- 모든 화면에서 Home 버튼으로 즉시 초기 화면 복귀
- 무입력 자동 복귀로 다음 사용자를 위한 초기화 보장
- 상태 전환 시마다 무입력 타이머 자동 리셋

🏗️ 핵심 설계

MVP (Model-View-Presenter)

View가 MonoBehaviour인 Unity 특성에 맞춰 **Passive View 방식의 MVP**를 적용했습니다. View는 이벤트 발행만 수행하고, Presenter(State)가 이벤트를 구독하여 로직을 처리합니다.

```
  View: event Action OnNextClicked  →  발행만
 State: _view.OnNextClicked += …  →  구독 + 전환 결정
```

- View는 Presenter를 모른다 → View 단독 교체·재사용 가능
- 데이터 바인딩은 `BindView()` 한 곳에 집중 → 변경 지점 명확

FSM (Finite State Machine)

각 화면을 State로 모델링하여 **5단계 라이프사이클**을 제공합니다.

```
Init → Enter → Update → Exit → Dispose
1회    매번    매프레임  이탈시   종료시
```

- `Init()` 1회 보장으로 이벤트 중복 구독 방지 (`HashSet` 으로 추적)
- 전환 로직이 State 안에 캡슐화 → 화면 추가 시 기존 코드 영향 없음

DataRepository

콘텐츠를 코드에서 분리하여 **CSV 파일로 외부화**했습니다. `DataConfig.json` 이 View-CSV 매핑을 선언합니다.

- 코드 변경 없이 콘텐츠 교체 가능
- View당 복수 CSV 지원 (후순위 파일이 동일 키 덮어씀)

🧰 기술 스택

분류	기술
엔진	Unity

분류	기술
언어	C#
UI	Unity uGUI (Canvas, Button, Image)
데이터 포맷	CSV (콘텐츠), JSON (설정)
JSON 파싱	JsonUtility (Unity 내장)
파일 I/O	System.IO (동기 읽기)
이벤트 시스템	C# event Action (델리게이트)
싱글톤	커스텀 MonoSingleton<T> (DontDestroyOnLoad)

⚙ 주요 시스템

NavigationManager (오케스트레이터)

StateMachine을 소유하고 State 등록, 초기 화면 진입, IdleManager 연동을 총괄하는 시스템 중앙 관리자.

StateMachine (상태 전환 엔진)

State 등록·전환·라이프사이클(Init/Enter/Exit/Dispose)을 관리. Init 호출 이력을 `HashSet` 으로 추적하여 1회 보장.

IdleManager (무입력 감지)

마우스·터치·키보드 입력을 통합 감지하고, 타임아웃 시 이벤트를 발행. Pause/Resume API로 팝업 등 예외 상황 대응.

DataRepository (데이터 저장소)

DataConfig.json 파싱 → CSV 로드 → PageData 변환을 앱 시작 시 1회 수행. View 타입 기반으로 데이터를 제공.

MonoSingleton<T> (싱글톤 베이스)

스레드 안전, DontDestroyOnLoad, 앱 종료 시 재생성 방지, 조건부 로그를 포함한 제네릭 싱글톤 베이스 클래스.

📊 데이터 흐름

```
 DataConfig.json (View-CSV 매핑 선언)
        ↓
CSVParser.Read() (StreamingAssets에서 동기 파싱)
        ↓
PageData (View별 key-value 데이터 컨테이너)
        ↓
State.BindView() (PageData → View UI 세팅)
        ↓
View.Show() (사용자에게 화면 표시)
```

👩‍💻 기술적 도전

1️⃣ 이벤트 중복 구독 버그 방지

문제점

FSM에서 화면을 재방문할 때마다 `Init()` 이 호출되면, 동일 이벤트에 대한 구독이 중복으로 쌓여 버튼 1번 클릭에 핸들러가 여러 번 실행되는 버그가 발생합니다.

해결

`StateMachine` 내부에 `HashSet<Type> _initializedStates` 를 두어 Init 호출 이력을 추적합니다. `HashSet.Add()` 가 `true` 를 반환할 때만(최초 진입) Init을 호출하여, 이벤트 구독이 정확히 1회만 수행되도록 보장했습니다.

2️⃣ 비활성 View의 Awake 미호출 문제

문제점

Unity에서 `GameObject.SetActive(false)` 인 오브젝트는 `Awake()` 가 호출되지 않습니다. Inspector에서 View를 비활성화해두면 버튼 바인딩, 초기 Hide 등 BaseView.Awake() 로직이 실행되지 않아 런타임 에러가 발생합니다.

해결

`NavigationManager.OnSingletonAwake()` 에서 `ActivateAllViews()` 를 호출하여 모든 View를 강제 활성화합니다. BaseView의 `_showOnAwake = false` 설정으로 활성화 직후 자동 Hide되므로, Awake 보장과 초기 비표시를 동시에 달성했습니다.

3️⃣ 콘텐츠 변경 시 재빌드 문제

문제점

화면 텍스트·이미지 경로가 코드에 하드코딩되어 있으면, 기획 변경 때마다 코드 수정 → 빌드 → 배포 사이클을 반복해야 합니다. 키오스크 현장에서의 즉각 반영이 불가능했습니다.

해결

`StreamingAssets` 에 CSV 파일로 콘텐츠를 분리하고, `DataConfig.json` 으로 View-CSV 매핑을 선언적으로 관리합니다. CSV 파일만 교체하면 빌드 없이 콘텐츠가 반영되며, `DataRepository` 가 앱 시작 시 자동으로 로드합니다.

⚖️ 설계 트레이드오프

선택	이유
동기 파일 I/O	Windows 전용 + 로컬 디스크 + 앱 시작 시 1회 로드 → 비동기의 복잡성 대비 실이익 없음
NavigationManager 싱글톤	단일 씬 구조에서 글로벌 접근 용이. 다중 씬 시 View 재참조 필요하지만 키오스크 환경에서는 제약 없음
State → NavigationManager 직접 참조	콜백 주입 대비 코드 간결. 테스트 Mock은 어렵지만 키오스크 환경에서 단위 테스트 요구 낮음
JsonUtility 사용	외부 라이브러리 의존성 제로. 고정 구조 JSON이므로 기능 충분
CSV key-value 형식	비개발자도 편집 가능한 단순 포맷. 복잡한 데이터 구조는 지원하지 않지만 키오스크 콘텐츠에는 적합

🎯 프로젝트 결과

- **재사용 가능한 키오스크 UI 프레임워크** 완성 — View + State + CSV 3개 파일만 추가하면 새 화면 동작
- **MVP + FSM + DataRepository** 패턴 조합으로 관심사 분리 달성 — View 교체 시 State 코드 변경 불필요
- **콘텐츠 외부화** — CSV 파일 교체만으로 화면 내용 변경, 빌드 없이 현장 반영 가능
- **이벤트 중복 구독 버그 원천 차단** — HashSet 기반 Init 1회 보장 메커니즘
- **무입력 자동 복귀** — 마우스·터치·키보드 통합 감지, Pause/Resume 지원

📌 배운 점

- **Unity에서의 MVP 적용**: MonoBehaviour 제약 속에서 View-Presenter 분리를 위해 이벤트 기반 Passive View 패턴이 효과적임을 확인
- **FSM 라이프사이클 설계**: Init/Enter/Exit/Dispose 분리로 이벤트 구독 타이밍 문제를 구조적으로 해결하는 방법 학습
- **외부 데이터 분리의 가치**: 코드와 콘텐츠를 분리하면 기획 변경 대응 속도가 크게 향상됨
- **단순함의 가치**: 키오스크 환경 특성(단일 씬, Windows 전용)을 파악하고 불필요한 복잡도(비동기, 다중 씬 대응)를 배제하는 것이 더 나은 설계

🔮 향후 개선

- **페이지 전환 애니메이션**: DoTween 연동으로 Fade/Slide 전환 효과 추가
- **동적 페이지 생성**: 고정 View 대신 Prefab 기반 동적 View 인스턴싱
- **분석 시스템 연동**: 화면별 체류 시간, 터치 빈도 등 사용 데이터 로깅
- **원격 콘텐츠 업데이트**: 서버에서 CSV/이미지를 다운로드하여 현장 방문 없이 콘텐츠 교체